数据库系统基础：概念、发展阶段与数据模型

一、核心基本概念（必考）

1. 数据

定义：描述事物的符号记录，是广义概念，包含数字、文本、图像、视频、语音、图形等。

2. 数据库（DB，Database）

定义：长期储存在计算机内、**有组织、可共享**的大量数据的集合。
核心特点：①长期存储 ②有组织 ③可共享（基本特点）；④冗余度小 ⑤数据独立性高 ⑥易扩展性（对比人工/文件管理的优势特点）。

3. 数据库管理系统（DBMS，Database Management System）

定位：位于用户和操作系统（OS）之间的大型复杂数据管理软件。
核心功能（5项）：

- 数据定义：提供**DDL（数据定义语言，Data Definition Language）**，定义数据库数据对象的组成与结构；
- 数据组织、存储与管理：管理数据字典、用户数据、存储路径，定义文件结构与存储方式；
- 数据操纵：提供**DML（数据操纵语言，Data Manipulation Language）**，实现数据的查询、删除、修改等操作；
- 事务与运行管理：统一管理数据库建立/运用/维护，保证数据安全性、完整性；
- 数据库建立与维护。

4. 数据库系统（DBS，Database System）

组成：$DBS = DB + DBMS$+应用程序+DBA（数据库管理员，Database Administrator）。

5. 计算机系统层次结构（由高到低）

用户 → 应用系统/开发工具 → DBMS（DBA参与管理） → 操作系统（OS） → 数据库（DB）。

二、数据管理技术的发展阶段（选择/填空高频）

共经历3个阶段，核心从**低共享、高冗余**向**高共享、低冗余**发展，关键对比4个维度：共享程度、独立性、结构化、数据控制能力，具体如下：

1. **人工管理阶段**：无专用存储设备（仅卡片/纸带）、无OS；数据处理为批处理；**数据不共享、冗余度极大、无独立性、无结构化设计。**
2. **文件系统阶段**：出现磁盘/磁鼓、有文件系统；处理方式为批处理+联机处理；**共享性弱、冗余度高、独立性低、记录内有结构但整体无。**
3. **数据库系统阶段**：出现DBMS；**共享性强、冗余度低且易扩充、独立性高、整体数据结构化**（核心优势阶段）。

三、数据库系统的核心特点（简答/选择）

1. 整体数据的结构化（最主要特征，与文件系统的核心区别）

- 文件系统：仅记录内部有结构，记录间联系固定，需程序员维护；
- 数据库系统：数据面向**整个组织**（非单一应用），存取方式灵活，可存取单个/一组数据项（文件系统仅能存取记录）。

2. 数据共享性强、冗余度低且易于扩充

- 共享性提升直接降低数据冗余，节省存储空间；
- 避免数据的不相容性与不一致性（如单处修改，全域同步）；
- 数据面向整个系统，可快速新增应用，扩展性强。

3. 数据独立性强（由DBMS的两级映像保障，核心考点）

分为物理独立性和逻辑独立性，定义如下：

- **物理独立性**：用户应用程序与数据的**物理存储**相互独立，数据存储方式/管理方式改变，应用程序无需修改；
- **逻辑独立性**：用户应用程序与数据库的**逻辑结构**相互独立，数据逻辑结构改变，应用程序无需修改。

4. 数据由DBMS统一管理和控制

实现数据的安全性、完整性保障，以及事务的正常运行。

四、数据模型（数据库系统的核心和基础，必考选择）

1. 基本定义

数据模型是对现实世界数据特征的**抽象与模拟**，用于描述、组织数据并对数据进行操作，是数据库系统的核心和基础。

2. 分类（按应用目的划分，2大类）

类型	建模视角	核心用途	典型代表
概念模型（信息模型）	用户观点	数据库设计	ER模型（实体-联系模型）
逻辑模型+物理模型	计算机系统观点	数据库管理系统（DBMS）实现	层次、网状、关系模型

3. 概念模型核心术语（必考，选择/填空/简答）

概念模型用于描述现实世界的事物及联系，核心术语及定义如下：

- **实体**：客观存在且相互区别的事物，可具体（学生、学院）或抽象（学生与学院的关系）；
- **属性**：实体所具有的某一特征，用于表征实体（如学生的学号、姓名、专业），一个实体可由多个属性刻画；
- **码**：唯一标识实体的**属性/属性集**（如学号是学生实体的码，姓名非码，因可能重名）；
- **实体型**：用**实体名+属性名集合**抽象刻画同类实体，格式：$\text{实体名}(\text{属性}_1,\text{属性}_2,...,\text{属性}_n)$（如学生(学号,姓名,专业,性别)）；

- **实体集**：同一类型实体的集合（如全体学生构成学生实体集）；
- **实体间的联系**：不同实体集之间的关联，分为3类：
    1. 一对一（1:1）：如我国婚姻关系（一个丈夫对应一个妻子，反之亦然）；
    2. 一对多（1:n）：如老师与学生（一个老师教多个学生，一个学生对应一个任课老师）；
    3. 多对多（m:n）：如学生与课程（一个学生选多门课，一门课被多个学生选）。
- **ER模型**：实体-联系（Entity-Relationship）模型，是概念模型的核心表示方法，期末高频考点（后续重点讲解）。

4. 常见逻辑模型（选择高频，重点掌握关系模型）

（1）层次模型

数据库中**最早出现**的逻辑模型，数据结构为树形结构，满足2个条件：
① 有且只有一个**根节点**；
② 根以外的其他节点有且只有一个**双亲节点。**
缺点：无法表征多双亲的复杂联系，贴合现实性差。

（2）网状模型

为解决层次模型缺陷提出，满足2个条件：
① 允许一个及以上节点**无双亲**；
② 一个节点可以有**多于一个的双亲。**
优点：可描述更复杂的实体联系；**缺点**：结构复杂，不易维护。

（3）关系模型（核心，后续章节重点）

核心特点：建立在严格的数学概念基础上，**数据结构为规范化的二维表。**
二维表核心术语：

- 关系：对应一张二维表；
- 元组：表中的**一行**为一个元组（对应实体集中的一个实体）；
- 属性：表中的**一列**为一个属性（对应实体的一个特征）；
- 码：唯一确定一个元组的属性/属性集（同概念模型的码）；
- 域：一组具有**相同数据类型**的值的集合（属性的取值范围）；
- 分量：元组中的一个**属性值**（如元组中“张三”为姓名属性的分量）；
- 关系模式：对关系的描述，格式：$关系名(属性_1,属性_2,...,属性_n)$（同实体型格式）。

五、数据库的三级模式与两级映像（重中之重，必考选择/简答）

是数据库系统实现**数据独立性**的核心机制，先掌握2个基础概念，再理解三级模式和两级映像。

1. 基础概念：型与值、模式与实例

- **型**：对一类数据的**结构和属性的说明**（如学生(学号,姓名,专业)）；
- **值**：型的**具体赋值**（如学生(20250001,张三,计算机)）；
- **模式**：数据库中全体数据的**逻辑结构和特征的描述**，是**型**的描述，不涉及具体值，相对稳定；
- **实例**：模式的**具体值**，是数据库中实际存储的数据，随数据更新而动态变化，一个模式可对应多个实例。

2. 三级模式（数据库系统的三个抽象层次，由外到内）

（1）外模式（子模式/用户模式）

- **定义**：描述数据的**局部逻辑结构**，是数据库用户能够看见和使用的局部数据的逻辑视图；
- **核心特点**：一个数据库**可以有多个外模式**（不同用户/应用对应不同局部视图）；
- **作用**：保证数据库安全性，用户仅能访问自身外模式中的数据，无法查看其他数据。
- **定义方式**：通过DDL严格定义。

（2）模式（概念模式）

- **定义**：描述数据库中**全体数据的逻辑结构和特征**，是数据库的核心抽象层，介于外模式和内模式之间；
- **核心特点**：一个数据库**只有一个模式**，是所有用户的公共数据视图。

（3）内模式（存储模式）

- **定义**：描述数据的**物理结构和存储方式**，是数据在数据库内部的实际存储形式；
- **具体内容**：包括数据的存储结构（如顺序/索引存储）、是否压缩/加密、记录格式（定长/变长）、记录是否跨物理块存储等；
- **核心特点**：一个数据库**只有一个内模式**。

3. 两级映像（实现数据独立性的关键，简答必考）

映像为外模式、模式、内模式之间的**转换规则**，由DBMS实现，分为外模式-模式映像和模式-内模式映像，分别保障逻辑独立性和物理独立性。

（1）外模式-模式映像

- **对应关系**：一个模式对应**多个**外模式-模式映像（一个模式对应多个外模式）；
- **作用**：**保证数据的逻辑独立性**；
- **原理**：当**模式（全体数据逻辑结构）**发生改变时，DBA修改对应的外模式-模式映像，使**外模式保持不变**；而应用程序基于外模式编写，因此应用程序无需修改，实现数据与程序的逻辑独立性。

（2）模式-内模式映像

- **对应关系**：一个数据库**只有一个**模式-内模式映像（一个模式+一个内模式）；
- **作用**：**保证数据的物理独立性**；
- **原理**：当**内模式（数据物理存储结构）**发生改变时，DBA修改模式-内模式映像，使**模式保持不变**；模式不变则外模式不变，应用程序无需修改，实现数据与程序的物理独立性。

4. 三级模式+两级映像的核心作用

通过**三级抽象层次**屏蔽了数据物理存储和整体逻辑结构的变化对应用程序的影响，通过**两级映像**实现了**物理独立性和逻辑独立性**，最终让用户/应用程序只需关注自身的外模式，无需关注数据的底层存储和整体逻辑，简化了应用开发与维护。